

UTK-ASL Digital Twin Predictive Maintenance Dataset

Complete Dataset Understanding, Directory Map & Recommended Project Flow

Machine-specific motor testbed • Bently Nevada System1 • Cutsforth InsightCM

EXECUTIVE TAKEAWAY

This repository is not one clean ML table. It is a research data archive containing motor-testbed measurements from two monitoring platforms, with two data representations: compact CSV summaries and high-resolution InsightCM JSON event records. For our project, we should NOT use every file. Start with two curated CSVs—CombinedFaults_02222025.csv and final.csv—understand the labels and timestamps, build the baseline machine-state model, and only then open the 24 split JSON files if raw/high-resolution analysis is required.

1. What exactly is this dataset?

The UTK-ASL repository is the experimental dataset released with the 2026 paper "Cross-domain digital twin architecture for predictive maintenance via machine learning and Large Language Models." It contains vibration/temperature monitoring data from motor testbeds under healthy baseline and faulty operating conditions. The data are organized around two monitoring platforms: Bently Nevada System1 and Cutsforth InsightCM. The repository documentation reports two motors of the same model and two monitoring platforms, enabling comparative/cross-platform research.

- This is machine-specific: the target asset is a motor testbed, not an abstract "factory machine".
- It contains physical fault conditions rather than only a generic binary failure flag.
- It has time-series information, multiple sensor modalities, and two monitoring-platform representations.
- The repository is designed for fault detection, condition monitoring, Digital Twin validation, streaming analytics, and cross-platform comparison.
- Important: the seven states are operating/fault conditions, not a guaranteed chronological degradation sequence. Do not interpret them as a natural healthy→failure trajectory.

2. What is inside the repository?

```
Dataset_digital_twin_predictive_maintenance/
├── README.md
├── data/
│   ├── BentlyNevada_System1/
│   │   ├── README.md
│   │   ├── s1_testbed_data.csv
│   │   ├── CombinedFaults_02222025.csv
│   │   ├── 01232025_Baseline/
│   │   ├── 01302025_Baseline/
│   │   ├── 01222025_EccentricRotor/
│   │   ├── 02052025_BentShaft/
│   │   ├── 02132025_FaultedBearing/
│   │   ├── 02172025_FaultedCoupling/
│   │   ├── 02172025_Imbalance/
│   │   └── CombinedFiles/
```

```

├── Combined_B1.csv
├── Combined_B2.csv
├── Combined_EC.csv
├── Combined_FB.csv
├── Combined_FC.csv
├── Combined_BS.csv
├── Combined_IM.csv
├── Cutsforth_InsightCM/
│   ├── README.md
│   ├── final.csv
│   ├── icm_chiller.json
│   ├── icm_chiller_split_10mb/
│   │   ├── icm_chiller_part001.json
│   │   ├── ...
│   │   └── icm_chiller_part024.json
│   └── image/
├── paper/
│   ├── 1-s2.0-S0360835226001154-main.pdf
│   └── readme.md
└── LICENSE

```

3. Directory-by-directory explanation

Path	What it means	Our use
README.md	Repository-level guide; explains the dataset, intended uses, structure, quick-start loading, reconstruction, limitations, and citation.	Read once; keep as reference.
data/	Main data area.	Everything used for modeling/analysis comes from here.
BentlyNevada_System1/	CSV-based monitoring data from the Bently Nevada System1 platform.	Use for compact multi-fault analysis and a strong baseline.
Cutsforth_InsightCM/	Higher-resolution multi-sensor monitoring data from InsightCM.	Use for dense time-series, sensor fusion, and streaming-style experiments.
paper/	Published research paper and notes.	Use to understand the original experiment and avoid accidentally reproducing the published contribution.
LICENSE	Repository license information.	Check before redistribution; repository states MIT licensing.

4. Bently Nevada System1 — complete map

System1 is the easier side of the repository. The official README describes approximately 15,000 records, two accelerometer channels, roughly one-minute sampling, and six metrics per channel: bias, derived peak, direct, RMS, velocity peak, and velocity RMS.

Item	One-line explanation	Use?
s1_testbed_data.csv	Testbed/configuration summary; describes the equipment/setup rather than being the primary ML table.	YES — documentation
CombinedFaults_02222025.csv	Master System1 CSV combining the fault conditions into one analysis-ready file.	YES — primary System1 file
01232025_Baseline/	First healthy/baseline measurement run.	YES — healthy

Item	One-line explanation	Use?
		reference
01302025_Baseline/	Second healthy/baseline measurement run.	YES — baseline variability
01222025_EccentricRotor/	Experiment associated with eccentric-rotor fault condition.	YES — fault class
02052025_BentShaft/	Experiment associated with bent-shaft fault condition.	YES — fault class
02132025_FaultedBearing/	Experiment associated with bearing fault condition.	YES — fault class
02172025_FaultedCoupling/	Experiment associated with coupling/misalignment fault condition.	YES — fault class
02172025_Imbalance/	Experiment associated with rotor-imbalance condition.	YES — fault class
CombinedFiles/	Pre-aggregated per-state CSV files, one for each baseline/fault condition.	OPTIONAL — useful for state-wise analysis
Combined_B1.csv	Combined Baseline 1 measurements.	Optional
Combined_B2.csv	Combined Baseline 2 measurements.	Optional
Combined_EC.csv	Combined eccentric-rotor measurements.	Optional
Combined_FB.csv	Combined faulted-bearing measurements.	Optional
Combined_FC.csv	Combined faulted-coupling measurements.	Optional
Combined_BS.csv	Combined bent-shaft measurements.	Optional
Combined_IM.csv	Combined imbalance measurements.	Optional

5. The seven System1 states

State	Meaning	Role in our Digital Twin
Baseline 1	Healthy reference run.	Normal-state calibration.
Baseline 2	Second healthy reference run.	Check normal-condition variability.
Eccentric Rotor	Rotor operates with eccentricity.	Fault diagnosis / signature learning.
Bent Shaft	Shaft is bent.	Fault diagnosis / signature learning.
Faulted Bearing	Bearing fault condition.	Fault diagnosis / maintenance reasoning.
Faulted Coupling	Coupling/misalignment condition.	Fault diagnosis / maintenance reasoning.
Imbalance	Rotor imbalance condition.	Fault diagnosis / maintenance reasoning.

6. Cutsforth InsightCM — complete map

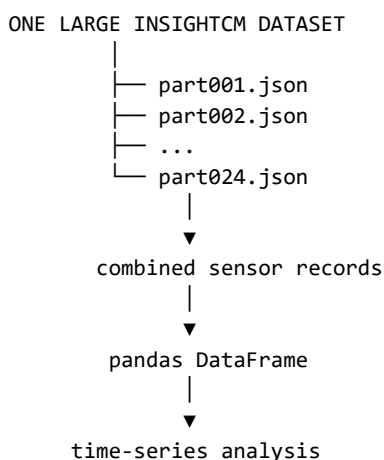
InsightCM is the more confusing side because the same underlying measurements are represented as a processed CSV and as high-resolution JSON records. The repository reports 852,860 discrete sensor readings, 4 accelerometers, 4 thermocouples, and 1 tachometer, with approximately 1–5 second temporal resolution.

Item	One-line explanation	Our use
final.csv	Processed/summary CSV recommended by the repository for most	PRIMARY InsightCM

Item	One-line explanation	Our use
	analysis; much easier than raw JSON.	file
icm_chiller.json	Single reconstructed high-resolution JSON representation of the InsightCM measurements; about 244 MB.	Do not start here; use only when raw records are required.
icm_chiller_split_10mb/	Directory containing the large JSON split into 24 roughly 10-MB chunks for Git-friendly distribution.	Use only for raw/high-resolution work.
icm_chiller_part001.json ... part024.json	Chunks of the same large JSON dataset; NOT 24 separate experiments.	Concatenate/read sequentially only if needed.
image/	Supporting images/figures associated with the dataset.	Reference only; not a training source.

7. What are the JSON files actually doing?

The JSON is a storage representation of the InsightCM event/sensor records. The 24 part files are simply a Git-friendly split of one large data collection. They should not be interpreted as 24 machines, 24 fault types, or 24 experiments.



The repository also provides a reconstruction approach: load parts 001–024, extend the record list, and optionally write one reconstructed JSON file. For most project work, however, the repository explicitly recommends using `final.csv` instead.

8. CSV vs JSON — which should we use?

Representation	Strength	Weakness	Decision
System1 CombinedFaults_02222025.csv	Compact, labeled, easy to model, fault-oriented.	Lower temporal resolution; fewer sensor modalities.	START HERE for baseline/fault classifier.
System1 CombinedFiles/*.csv	Convenient state-by-state analysis.	Duplicates the information in the master/combined view.	Use selectively.
InsightCM final.csv	Small, processed, easier multi-sensor analysis.	May not retain every raw event detail.	START HERE for InsightCM.
InsightCM split JSONs	High-resolution/raw event representation; richest source for custom temporal processing.	Large, nested, harder to parse; easy to misuse.	Use later, only if needed.
icm_chiller.json	One-file raw/high-resolution	Large (~244 MB), unnecessary for	Usually ignore

Representation	Strength	Weakness	Decision
	representation.	initial work.	initially.

9. The most important conceptual distinction

DO NOT THINK: 24 JSON files = 24 datasets.

THINK: 24 JSON files = one large InsightCM record set split into manageable chunks.

DO NOT THINK: Baseline → eccentric rotor → bent shaft → bearing → coupling → imbalance is a degradation timeline.

THINK: these are separate labeled operating/fault conditions. They are excellent for diagnosis/classification, but they do not by themselves establish a true chronological RUL trajectory.

10. Recommended project approach — the clean flow

PHASE 0 — REPOSITORY UNDERSTANDING

↓

Read README + platform READMEs + paper

↓

PHASE 1 — DATA AUDIT

↓

Load ONLY:

- System1/CombinedFaults_02222025.csv
- InsightCM/final.csv

↓

Inspect columns, timestamps, labels, missingness, sampling intervals

↓

PHASE 2 — LABEL & SENSOR MAPPING

↓

Map:

sensor → physical quantity → platform → timestamp → state/fault

↓

PHASE 3 — HEALTHY BASELINE

↓

Use Baseline 1 + Baseline 2 to learn normal machine behaviour

↓

PHASE 4 — FEATURE ENGINEERING

↓

Vibration: RMS, peak, std, kurtosis, skewness, crest factor, peak-to-peak

Temperature: mean, trend, variance, rate of change

Speed: mean, variance, stability

Cross-sensor: correlations, vibration magnitude, temp-vibration coupling

↓

PHASE 5 — BASELINE ML

↓

Random Forest / XGBoost / SVM

Task: 7-state fault diagnosis

↓

PHASE 6 — DIGITAL TWIN STATE

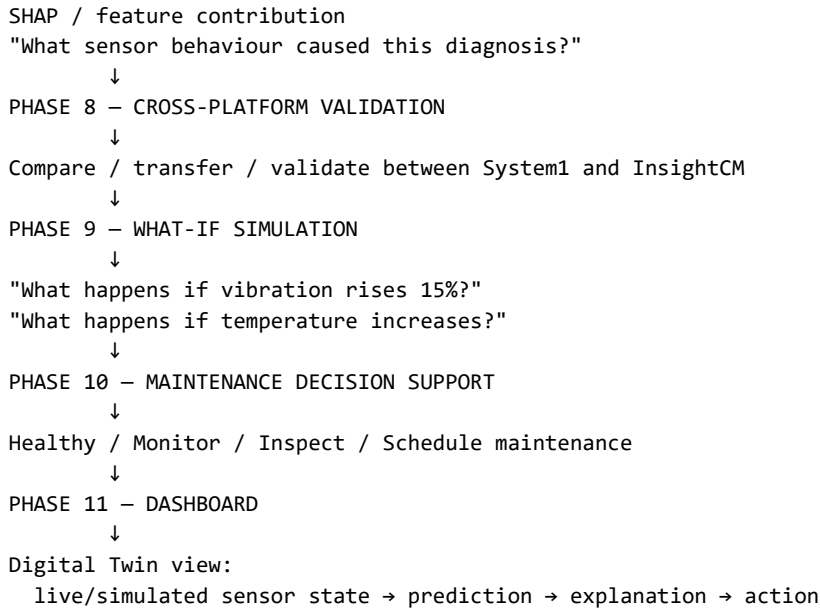
↓

Current machine state + health index + anomaly score + confidence

↓

PHASE 7 — EXPLAINABILITY

↓



11. What should be our primary research problem?

The strongest use of this dataset is not simply "predict machine failure." A better framing is: build a machine-specific Digital Twin state representation that fuses vibration, temperature and rotational information, diagnoses known motor fault states, quantifies machine health/anomaly level, explains the diagnosis, and evaluates robustness across two monitoring platforms.

Component	Proposed implementation	Why it matters
Machine state	7-class state representation with healthy baselines and five faults.	Represents a specific motor testbed rather than a generic machine.
Health index	0–100 composite health score calibrated from normal-state distance + fault probability.	Provides a continuous Digital Twin state variable; clearly document that it is a derived index, not a measured physical quantity.
Anomaly score	Distance from healthy baseline in feature space or reconstruction error.	Supports early warning before hard classification.
Explainability	SHAP on the classifier; feature-level explanations.	Turns a prediction into maintenance-relevant evidence.
Cross-platform validation	Compare predictions/feature behaviour across System1 and InsightCM after careful normalization/alignment.	A strong research angle enabled by this repository.
What-if layer	Perturb selected sensor features within physically plausible ranges and recompute model state.	Moves the project toward decision support rather than pure classification.
Dashboard	Sensor state + fault + health + explanation + maintenance action.	Makes the Digital Twin operationally understandable.

12. Exact workflow we should follow in implementation

1. Clone/download the repository and preserve the original directory structure as read-only raw data.
2. Create a separate project data layer. Never edit raw files directly.
3. Load CombinedFaults_02222025.csv and final.csv into separate DataFrames.
4. Print shape, column names, data types, timestamp ranges, unique labels, missing-value counts, duplicate timestamps, and sampling intervals.

5. Create a data dictionary mapping every useful column to sensor type, physical meaning, unit (when documented), platform, and role.
6. Normalize timestamps to a single timezone/format without inventing synchronization where it does not exist.
7. Build the System1 baseline first. This gives the team a simple, auditable fault-classification pipeline.
8. Build the InsightCM pipeline second. Preserve its time-series nature and test window-based features.
9. Use the two baseline runs to quantify normal-condition variability before defining health thresholds.
10. Train a baseline classifier with a leakage-safe split. Do NOT randomly split adjacent time samples if that would put nearly identical windows in train and test.
11. Evaluate using macro-F1, per-class precision/recall, confusion matrix, balanced accuracy and ROC-AUC where appropriate—not accuracy alone.
12. Add SHAP explanations and verify that the dominant features are physically plausible.
13. Construct the Digital Twin state layer from sensor state + diagnosis + health/anomaly score + confidence + timestamp.
14. Only after both pipelines work independently, attempt cross-platform alignment/validation.
15. Only use the raw JSON chunks if the processed CSV cannot answer a research question.
16. Build the dashboard last, after the analytical pipeline is stable.

13. Data leakage and validation rules

- Do not randomly split highly adjacent time-series rows into train/test; this can inflate performance.
- Prefer run-wise, time-wise, or condition-aware splits where possible.
- Do not train on the same experimental run and claim generalization to a new run without a clear protocol.
- Do not treat multiple JSON chunks as independent samples; they are pieces of the same underlying collection.
- Do not compare raw System1 and InsightCM numerical values as if their sensors/processing chains were identical. Normalize and document platform differences first.
- Do not call a model probability a physical health measurement. A health index must be explicitly defined as a derived indicator.
- Do not claim RUL unless the data provide a defensible degradation trajectory and failure-time target.

14. What we should NOT use initially

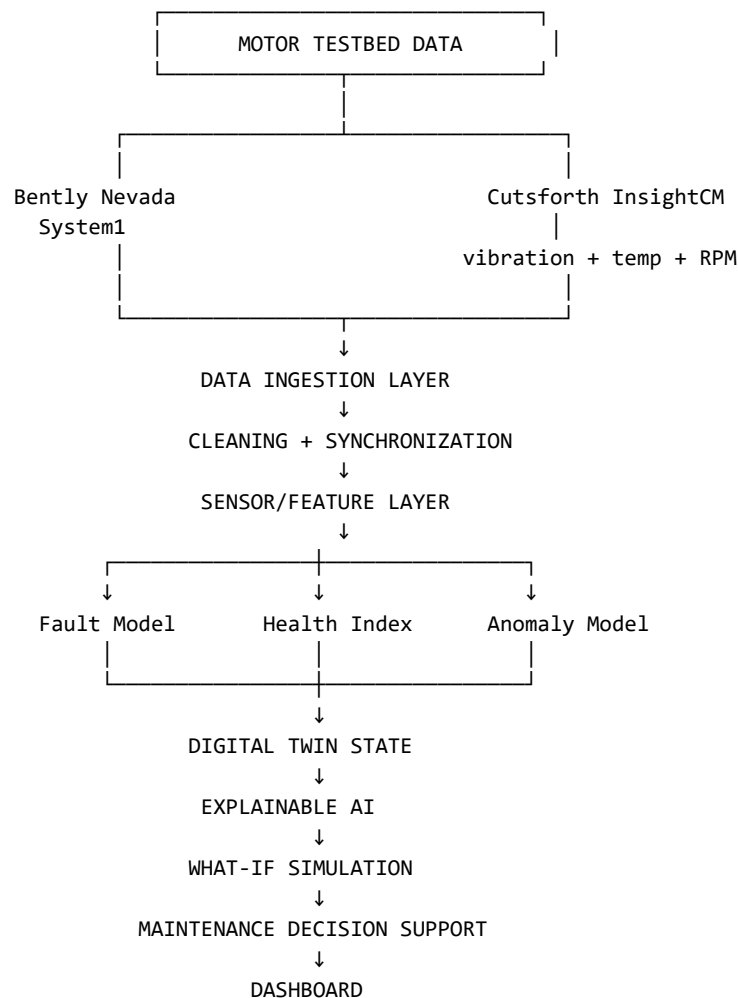
File / Folder	Why not initially?
icm_chiller.json	Large raw representation; unnecessary until we prove final.csv is insufficient.
icm_chiller_split_10mb/*.json	Same reason; 24 chunks are one underlying record set, not separate datasets.
image/	Supporting figures, not model input.
Every dated System1 folder simultaneously	Creates unnecessary preprocessing complexity; use master combined CSV first.
Every CombinedFiles CSV simultaneously	Mostly redundant for the first pipeline; use them for state-specific inspection when needed.

15. What questions this dataset can answer

Question	Can dataset support it?	Comment
Which motor fault is present?	YES	Strong classification use case.
What sensor behaviour distinguishes faults?	YES	Excellent for feature analysis/XAI.

Question	Can dataset support it?	Comment
What is normal motor behaviour?	YES	Two baseline measurements help quantify baseline variation.
Can vibration + temperature + speed improve diagnosis?	YES	Especially with InsightCM.
Can we build a machine-specific Digital Twin state?	YES	Strong conceptual fit.
Can we compare two monitoring platforms?	YES	One of the dataset's intended uses.
Can we do real-time/streaming simulation?	YES	High-resolution InsightCM records support this.
Can we prove true RUL?	NOT DIRECTLY	No clear run-to-failure degradation trajectory across these labeled states.
Can we claim universal machine generalization?	NO	Single testbed / limited conditions.
Can we simulate arbitrary physical operating conditions?	LIMITED	Only data-supported perturbations should be used unless a physics model is added.

16. Recommended final system architecture



17. Proposed project modules

Module	Input	Output
M1 — Data ingestion	Raw/curated CSVs	Standardized tables
M2 — Data audit	Standardized tables	Data quality report
M3 — Sensor mapping	Columns + platform docs	Machine data dictionary
M4 — Feature engineering	Sensor time series	Feature matrix/windows
M5 — Fault diagnosis	Features	Fault class + probability
M6 — Anomaly detection	Healthy baseline + windows	Anomaly score
M7 — Health index	Diagnosis + anomaly + sensor state	0–100 derived health indicator
M8 — Explainability	Trained model	SHAP/global/local explanations
M9 — Cross-platform validation	System1 + InsightCM	Robustness/comparison results
M10 — What-if engine	Current feature state	Counterfactual predictions
M11 — Decision support	Twin state + explanations	Maintenance recommendation
M12 — Dashboard	All outputs	Interactive Digital Twin UI

18. A practical folder structure for OUR project

```

digital_twin_motor/
├── data/
│   ├── raw/           # untouched downloaded repository
│   ├── interim/       # cleaned/intermediate files
│   └── processed/     # modeling-ready datasets
├── notebooks/
│   ├── 01_repository_audit.ipynb
│   ├── 02_system1_eda.ipynb
│   ├── 03_insightcm_eda.ipynb
│   ├── 04_feature_engineering.ipynb
│   ├── 05_fault_model.ipynb
│   ├── 06_health_anomaly.ipynb
│   ├── 07_xai.ipynb
│   └── 08_cross_platform.ipynb
├── src/
│   ├── data_loader.py
│   ├── preprocessing.py
│   ├── features.py
│   ├── models.py
│   ├── health_index.py
│   ├── explainability.py
│   ├── what_if.py
│   └── evaluation.py
├── dashboard/
│   └── app.py
├── models/
├── reports/
└── README.md

```

19. Final recommendation — how we should handle the confusing repository

USE NOW → BentlyNevada_System1/CombinedFaults_02222025.csv + Cutsforth_InsightCM/final.csv

USE FOR REFERENCE → s1_testbed_data.csv + platform READMEs + paper

USE SELECTIVELY → CombinedFiles/*.csv for fault-by-fault inspection

USE LATER → icm_chiller_split_10mb/*.json when we need raw/high-resolution records

IGNORE INITIALLY → icm_chiller.json and image/ for the first pipeline

Recommended order: System1 baseline → InsightCM time-series → sensor/feature mapping → fault diagnosis → health/anomaly layer → SHAP → cross-platform validation → what-if simulation → maintenance decision support → dashboard.

20. Source and citation notes

Primary source: UTK-ASL, Dataset: Digital Twin Predictive Maintenance, repository README and platform documentation. The repository states that the dataset supports the 2026 Computers & Industrial Engineering paper by Williams et al. The repository also provides the detailed directory structure, data overview, recommended CSV entry points, JSON reconstruction method, and limitations.

Repository: https://github.com/UTK-ASL/Dataset_digital_twin_predictive_maintenance

Paper: <https://doi.org/10.1016/j.cie.2026.111914>

Important published baseline: The 2026 paper already presents a cross-domain Digital Twin architecture using ML/LLM components. Our implementation should therefore be framed as an extension/research variant, not a reproduction.